

SISTEM CREDITE IPOTECARE

1. Crearea unei vizualizări complexe. Dați un exemplu de operație LMD permisă pe vizualizarea respectivă și un exemplu de operație LMD nepermisă.

-- Fisa de performanta a fiecărei sucursale

```
CREATE OR REPLACE VIEW v_performanta_sucursale AS
SELECT
    s.sucursala_id,
    s.numa_sucursala,
    s.oras,
    COUNT(DISTINCT a.angajat_id) AS numar_angajati,
    SUM(cr.sold_curent) AS total_credite_active
FROM sucursala s
JOIN angajat a      ON s.sucursala_id = a.sucursala_id
JOIN cerere_credit cc ON a.angajat_id = cc.angajat_id
JOIN contract con    ON cc.cerere_id = con.cerere_id
JOIN credit cr        ON con.contract_id = cr.contract_id
WHERE cr.stare_credit = 'Activ'
GROUP BY s.sucursala_id, s.numa_sucursala, s.oras;
```

-- OPERAȚIE PERMISĂ: Filtrarea sucursalelor performante

```
SELECT nume_sucursala, total_credite_active
FROM v_performanta_sucursale
WHERE total_credite_active > 100000;
```

-- OPERAȚIE NEPERMISĂ: Încercarea de a șterge o sucursală din View

```
DELETE FROM v_performanta_sucursale
WHERE sucursala_id = 1;
```

1. Formulați în limbaj natural și implementați în SQL: o cerere ce utilizează operația outer-join pe minimum 4 tabele, o cerere ce

utilizează operația division și o cerere care implementează analiza top-n.

--outer join pentru clienti si cereri

SELECT

c.num_e_denumire **AS** nume_client,
cc.cerere_id,
cc.status_cerere,
con.nr_contract,
cr.credit_id,
cr.stare_credit

FROM client c

LEFT OUTER JOIN cerere_credit cc **ON** c.client_id = cc.client_id

LEFT OUTER JOIN contract con **ON** cc.cerere_id = con.cerere_id

LEFT OUTER JOIN credit cr **ON** con.contract_id =
cr.contract_id

ORDER BY c.num_e_denumire;

--top 3 evaluatori in functie de valoarea evaluarilor

SELECT * FROM (

SELECT

e.num_e_evaluator,
e.nr_authorized,
r.valoare_estimata,
r.data_evaluare

FROM evaluator e

JOIN rapoarte_evaluare r **ON** e.evaluator_id = r.evaluator_id

ORDER BY r.valoare_estimata **DESC**

)

WHERE ROWNUM <= 3;

*--division, selecteaza creditele pentru care au fost folosite
toate metodele de plata*

SELECT cr.credit_id, cr.sold_curent

FROM credit cr

WHERE NOT EXISTS (

SELECT DISTINCT metoda_plata

FROM plata

MINUS

SELECT p.metoda_plata

FROM plata p

```

WHERE p.credit_id = cr.credit_id
);

```

1. 1. Aplicarea denormalizării, justificând necesitatea acesteia.

În faza de proiectare conceptuală, pentru a respecta Forma Normală 3 (FN3), am evitat stocarea valorilor derivate (calculate). Totuși, într-un sistem bancar de producție, interogarea repetată a valorii totale a garanțiilor imobiliare asociate unui credit implică interogări costisitoare între tabelele CREDIT, GARANTII, PROPRIETATE și RAPOARTE_EVALUARE. Pentru a optimiza am decis aplicarea unei denormalizări prin introducerea coloanei valoare_totala_proprietati direct în tabelul CREDIT.

```

-- 1. Modificare tabel
ALTER TABLE credit ADD valoare_totala_proprietati NUMBER(12,2)
      DEFAULT 0;

-- 2. Actualizarea datelor existente
UPDATE credit cr
SET cr.valoare_totala_proprietati = (
    SELECT NVL(SUM(re.valoare_estimata), 0)
    FROM garantii g
    JOIN proprietate p ON g.proprietate_id = p.proprietate_id
    JOIN rapoarte_evaluare re ON p.proprietate_id =
        re.proprietate_id
    WHERE g.credit_id = cr.credit_id
);

```

1. Exemplificarea isolation levels prin exemple de tranzacții în paralel

Luam 2 tranzacții concurente: * **Tranzacția 1 (T1 - Angajatul):** Interoghează soldul curent al unui credit pentru a genera un raport intern. * **Tranzacția 2 (T2 - Clientul):** Efectuează o plată online, scăzând soldul creditului cu 1.000 RON.

1. T1 (Angajatul) pornește și citește soldul inițial:

```

SELECT sold_curent FROM credit WHERE credit_id = 101;

```

2. T2 (Clientul) face plata în aceeași secundă (în alt terminal/ sesiune):

```

sql UPDATE credit SET sold_curent = sold_curent -
1000 WHERE credit_id = 101; COMMIT;

```

3. T1 (Angajatul) citește din nou soldul în cadrul tranzacției sale:

```
sql SELECT sold_curent FROM credit WHERE credit_id = 101;
```

- **Scenariul 1:** Utilizarea nivelului „READ COMMITTED”
sql SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

- T1 citește initial valoarea soldului
- T2 modifica valoarea soldului
- T1 citește valoarea modificata

READ COMMITTED previne Dirty Read dar permite Non-repeatable Read(valoarea soldului este modificata in timpul altei tranzactii).
Tranzactiile nu se blocheaza una pe cealalta.

- **Scenariul 2:** Utilizarea nivelului „SERIALIZABLE”
sql SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; SERIALIZABLE
previne atat Dirty Read cat si Non-repeatable Read. T1 vede doar un snapshot al bazei de date. Daca insa T1 incearca sa modifice valorile, tranzactia ar fi blocata.